

Quotienting Synthesised Programs: Term Rewriting, E-Graphs, and Equality Saturation

Yiding Song

April 29, 2026

Motivation

This document is a theoretical companion to the enumeration-RL document. It addresses an open problem flagged in §A.3 of that document:

Some RL-generated programs suffer from internal redundancies. While their overall behaviour is interesting, they may be represented by shorter programs. For example, `product (repeat 10 8)` is a constant internal node in the AST. In the future, it may be worth simplifying RL-generated programs.

The pipeline currently deduplicates programs by their *observational* behaviour: two programs p, q with the same fingerprint $\Phi(p) = \Phi(q)$ on the test suite are considered equivalent. This catches programs that compute the same overall function, but is blind to redundancy *inside* a program. Two programs may have distinct fingerprints (and so both be retained in the corpus) while one of them contains a sub-tree that is itself constant or otherwise reducible.

We want a transformation $\mathcal{Q} : \text{programs} \rightarrow \text{programs}$ such that:

1. $\Phi(\mathcal{Q}(p)) = \Phi(p)$ on the test suite (behaviour preserved);
2. $|\mathcal{Q}(p)| \leq |p|$ in the size metric of §2.1 of the enumeration document (programs only shrink);
3. $\mathcal{Q}$ is well-defined on a quotient: programs in the same equivalence class map to a common canonical representative.

The remainder of this document develops the machinery from the programming-languages literature that addresses this problem rigorously: equational theories and term rewriting (§1), the phase-ordering obstacle (§2), e-graphs as a non-destructive representation (§3), equality saturation as the algorithm that grows them (§4), extraction as the optimisation pass that picks shortest representatives (§5), and a concrete instantiation against the MLMP DSL (§6). We close with how this slots into the existing pipeline (§7), adjacent techniques (§8), and a summary (§9).

1 Equational Theories and Term Rewriting

1.1 Signatures and Term Algebras

A **many-sorted signature** is a pair $\Sigma = (S, F)$ where S is a set of sorts and F is a set of function symbols, each $f \in F$ equipped with an arity $f : s_1 \times \cdots \times s_k \rightarrow s_0$ with $s_i \in S$. In our

setting $S = \{\text{int}, \text{bool}, \text{list}[\text{int}], \dots\}$ as in §1.2 of the enumeration document, and F is the set of DSL primitives.

Given a sort-respecting set of variables $X = \bigsqcup_{s \in S} X_s$, the **term algebra** $T(\Sigma, X)$ is built inductively: every $x \in X_s$ is a term of sort s ; if $f : s_1 \times \dots \times s_k \rightarrow s_0$ and each t_i has sort s_i , then $f(t_1, \dots, t_k)$ is a term of sort s_0 .

A **substitution** is a sort-preserving map $\theta : X \rightarrow T(\Sigma, X)$, lifted homomorphically: $\theta(f(t_1, \dots, t_k)) = f(\theta(t_1), \dots, \theta(t_k))$.

Positions in a term are sequences over $\mathbb{N}$: ε is the root, and if $t = f(t_1, \dots, t_k)$ then $i \cdot p$ addresses position p within t_i . We write $t|_p$ for the subterm at p and $t[s]_p$ for the term obtained by replacing $t|_p$ with s .

1.2 Equational Theories and the Word Problem

Given a set of axioms $E = \{l_i \approx r_i\}_{i \in I}$, the **equational theory** $\approx_E$ is the smallest Σ -congruence on $T(\Sigma, X)$ containing every instance $\theta(l_i) \approx \theta(r_i)$. Concretely, $s \approx_E t$ iff there exists a finite chain $s = u_0, u_1, \dots, u_n = t$ where each u_{j+1} is obtained from u_j by replacing some occurrence of $\theta(l)$ by $\theta(r)$ (or vice versa) for some axiom $l \approx r$.

The **word problem** for E asks: given s, t , decide whether $s \approx_E t$. It is undecidable for arbitrary E but tractable for the systems we consider.

1.3 Rewrite Rules and Reduction

A **rewrite system** R is a set of oriented pairs $\{l \rightarrow r\}$. The single-step rewrite relation $\rightarrow_R$ is defined by

$$s \rightarrow_R t \iff \exists p, l \rightarrow r \in R, \theta : s|_p = \theta(l) \text{ and } t = s[\theta(r)]_p.$$

Its reflexive-transitive closure is denoted $\rightarrow_R^*$. A term t is in **normal form** (with respect to R) if no rule applies anywhere in t .

1.4 Confluence, Termination, Canonical Forms

R is **terminating** iff no infinite chain $t_0 \rightarrow_R t_1 \rightarrow_R \dots$ exists.

R is **confluent** iff whenever $u \rightarrow_R^* s$ and $u \rightarrow_R^* t$, there exists v with $s \rightarrow_R^* v$ and $t \rightarrow_R^* v$. Equivalently (Church–Rosser), the symmetric closure of $\rightarrow_R^*$ coincides with $\approx_R$.

Newman’s lemma states that a terminating system is confluent iff it is *locally* confluent: divergences from a single step always rejoin. The point is that local confluence is decidable (via the *critical pair lemma*: check joinability of every pair of overlapping rule applications), so for terminating systems we can decide confluence mechanically.

Canonicalisation theorem. If R is confluent and terminating, every $\approx_R$ -equivalence class contains a unique normal form, and any reduction strategy reaches it. This is precisely the property we want from $\mathcal{Q}$: pick the (unique) normal form as the canonical representative.

1.5 Knuth–Bendix Completion

Knuth–Bendix completion (Knuth and Bendix, 1970) takes an equational theory E and tries to construct a confluent and terminating R presenting the same congruence:

1. Choose a **reduction order** $\succ$: a well-founded order on terms compatible with substitution and context.
2. Orient each $l \approx r$ as $l \rightarrow r$ if $l \succ r$, else $r \rightarrow l$ if $r \succ l$, else **fail**.
3. Compute **critical pairs** from overlapping rules.
4. Normalise each critical pair. If they reach the same normal form, discard. Otherwise add the new equation as a rule (re-orienting if needed).
5. Repeat until no new critical pairs.

Outcomes are *success* (a confluent and terminating R), *divergence* (the procedure runs forever), or *failure* (an unorientable equation appears). The canonical failure is commutativity $x + y \approx y + x$: neither side is smaller under any reasonable order. Equally, associativity-plus-commutativity (AC) generates infinitely many equivalent permutations, defeating naive completion.

This is the practical motivation for equality saturation: rather than committing to a confluent rewrite system up front, both directions of every equation are kept simultaneously inside an e-graph, and the choice of representative is deferred to extraction time.

1.6 Worked Example

Consider the signature with sort L (lists), constant **nil**, unary **reverse**, and binary **concat**. Take the rules

$$\begin{aligned}
 R_1 &: \text{reverse}(\text{nil}) \rightarrow \text{nil} \\
 R_2 &: \text{reverse}(\text{reverse}(x)) \rightarrow x \\
 R_3 &: \text{concat}(\text{nil}, x) \rightarrow x \\
 R_4 &: \text{concat}(x, \text{nil}) \rightarrow x.
 \end{aligned}$$

Reduce $\text{reverse}(\text{reverse}(\text{concat}(xs, \text{nil})))$ in two different orders:

Step	Trace A (outermost)	Trace B (innermost)
0	$\text{reverse}(\text{reverse}(\text{concat}(xs, \text{nil})))$	$\text{reverse}(\text{reverse}(\text{concat}(xs, \text{nil})))$
1	by R_2 : $\text{concat}(xs, \text{nil})$	by R_4 : $\text{reverse}(\text{reverse}(xs))$
2	by R_4 : xs	by R_2 : xs

Both traces reach the same normal form xs . To verify confluence in general one inspects every critical pair: the only nontrivial overlap is between R_1 and R_2 at $x = \text{nil}$, where both routes reach **nil**. Termination follows because every rule strictly decreases AST size. By Newman's lemma the system is confluent, so every term has a unique normal form reachable by any strategy.

2 Why Naive Rewriting Is Fragile

The previous example was lucky: the rules were locally confluent and strictly size-decreasing. Realistic DSLs are not so accommodating. Two pathologies arise even when each individual rule shrinks the program.

2.1 Phase Ordering

Consider the rules

$$\begin{aligned}\rho_1 &: \text{map } (\lambda x. x) \ xs \rightarrow xs \\ \rho_2 &: \text{reverse } (\text{reverse } xs) \rightarrow xs \\ \rho_3 &: \text{add } y \ 0 \rightarrow y\end{aligned}$$

applied to

$$t_0 = \text{map } (\lambda x. \text{add } x \ 0) \ (\text{reverse } (\text{reverse } xs)).$$

Different orderings produce different outcomes. Following an outermost-leftmost strategy:

Step	Term
0	$\text{map } (\lambda x. \text{add } x \ 0) \ (\text{reverse } (\text{reverse } xs))$
1	by ρ_2 : $\text{map } (\lambda x. \text{add } x \ 0) \ xs$
2	by ρ_3 (under λ): $\text{map } (\lambda x. x) \ xs$
3	by ρ_1 : xs

A strategy that does not look under lambdas, or that gives up after one full sweep finds no more matches at the top level, would stop at step 1 with size 7. The rule ρ_1 is *enabled by* the prior firing of ρ_3 inside the lambda body; missing this enablement leaves a redex on the table.

One can patch the strategy to keep iterating until no rule applies — but this only works for rewrite systems that strictly decrease some monotone measure on terms. As soon as bidirectional rules enter the picture, “iterate to fixpoint” loops forever.

2.2 Bidirectional Rules

Several useful equalities are not directional. For example,

$$x \cdot 2 \approx x + x.$$

Each side is preferable in a different context: $x \cdot 2$ is shorter (two AST nodes versus three for $x + x$) but $x + x$ might be cheaper to evaluate when x is already cached. Similarly, the fusion law $\text{map } f \ (\text{map } g \ xs) \approx \text{map } (\lambda x. f(g\ x)) \ xs$ shrinks the program in the right-to-left direction (one pass instead of two) but unfusing is sometimes useful for enabling further rewrites in the bodies of the inner maps.

If we orient such a rule one way, we lose the other. If we orient both ways, every rewriter that does not detect the cycle loops indefinitely. The fix — which is what equality saturation provides — is to give up on rewriting as a destructive operation and instead represent both sides of every equation simultaneously, deferring the choice of representative to extraction time.

3 E-Graphs

3.1 Definitions

Fix a signature Σ . An **e-node** is a pair $(f, [c_1, \dots, c_k])$ where $f \in F$ has arity k and each c_i is an **e-class identifier**. An **e-class** is a set of e-nodes deemed equivalent. An **e-graph** E is a set

of e-classes $\mathcal{C}$, together with a partition $\text{nodes}(E) = \bigsqcup_{c \in \mathcal{C}} c$ and a union–find structure uf over the e-class identifiers.

The crucial design choice: an e-node’s *children* are e-classes, not specific e-nodes. Alternative representations of the same sub-term inhabit a common e-class, and parents reference the e-class once. This is what makes the structure compact.

3.2 Hash Consing

Two e-nodes are *syntactically equal* iff they have the same operator and the same children list (up to canonical e-class identifiers). Hash consing maintains the invariant that syntactically equal e-nodes share an e-class. On insertion of a new e-node $n = (f, [c_1, \dots, c_k])$:

1. Canonicalise the children list via uf.find .
2. Look up n in a global hash table keyed by canonical form.
3. If present, return the existing e-class identifier.
4. Otherwise, create a fresh e-class containing n and return its identifier.

3.3 Congruence

The structural invariant of the e-graph is the **congruence axiom**: if two e-nodes have the same operator and pointwise-equal children classes, they are in the same e-class. Symbolically,

$$(f, [c_1, \dots, c_k]), (f, [c'_1, \dots, c'_k]) \in \text{nodes}(E) \text{ and } c_i \equiv c'_i \forall i \implies \text{they share an e-class.}$$

Maintaining this invariant is the job of a procedure **rebuild()** run after every union:

1. For every e-node whose children list contains a class that has been merged, re-canonicalise the list.
2. If the canonicalised e-node now collides with another e-node, union their e-classes.
3. Repeat until no more unions occur.

Without rebuild, asserting an equality at a sub-term would fail to propagate through the parent contexts that reference it.

3.4 Compactness

Suppose the e-graph contains G e-nodes representing G distinct sub-terms. An equation $a \approx b$ that licenses N rewrites in the original term language would expand to $G \cdot N$ distinct alternative terms if everything were materialised. In the e-graph, asserting $a \approx b$ is a constant-time union plus a sub-graph-bounded congruence cascade; the total e-graph size grows by $O(N)$, not $O(GN)$. This compactness is what makes equality saturation tractable for non-trivial rewrite systems.

3.5 Worked Example

Build the e-graph for $\{(a + b) \cdot c, (b + a) \cdot c\}$. After insertion (every e-node in its own singleton e-class):

```

C0:  var a
C1:  var b
C2:  var c
C3:  add(C0, C1)          -- represents (a + b)
C4:  add(C1, C0)          -- represents (b + a)
C5:  mul(C3, C2)          -- represents (a + b) * c
C6:  mul(C4, C2)          -- represents (b + a) * c

```

Now assert commutativity by adding the rewrite $\text{add}(x, y) \rightarrow \text{add}(y, x)$. The e-matcher finds matches at C_3 (with $\theta = \{x \mapsto C_0, y \mapsto C_1\}$) and at C_4 (symmetric). Firing the first match constructs $\text{add}(C_1, C_0)$, which already lives in C_4 ; we therefore union C_3 and C_4 . Now **rebuild** runs: the e-nodes $\text{mul}(C_3, C_2)$ and $\text{mul}(C_4, C_2)$ have congruent canonical children lists (both become $[\{C_3, C_4\}, C_2]$), so they collide and their e-classes are unioned. Final state:

```

C0:  var a
C1:  var b
C2:  var c
{C3, C4}:  add(C0, C1), add(C1, C0)
{C5, C6}:  mul({C3, C4}, C2)

```

Note: the multiplication e-class collapsed automatically as a downstream effect of the addition union, without any rewrite being applied at the multiplication level. This is the power of congruence closure — it lifts equalities through every parent context for free.

4 Equality Saturation

4.1 The Algorithm

Equality saturation grows the e-graph by repeatedly applying every rewrite rule everywhere it matches:

```

function saturate(initial_terms, R, budget):
  E <- empty e-graph
  for t in initial_terms: E.add(t)
  repeat:
    matches <- []
    for each rule l -> r in R:
      for each e-match sigma of l in E:
        matches.append( (r, sigma, eclass-of(sigma(l))) )
    for (r, sigma, c) in matches:
      c' <- E.add(sigma(r))
      E.union(c, c')
    E.rebuild()
    if no new equalities or budget exhausted: break
  return E

```

The two-phase structure — collect matches first, apply later — is essential. Applying matches as we find them would mean the new e-nodes interfere with ongoing matching within the same iteration, leading to schedule-dependent behaviour and potentially missed matches.

4.2 E-matching

E-matching is pattern matching against an e-graph rather than against a term: given a pattern l (a term with variables) and an e-graph E , find every substitution σ such that $\sigma(l)$ is represented

in E . The complication is that an e-class may contain multiple e-nodes that match different sub-patterns of l , and exploring all combinations is exponential in the worst case. In practice, sophisticated indexing structures (those described by [Willsey et al. \(2021\)](#)) make e-matching the practical bottleneck of the algorithm. We treat it as an oracle here.

4.3 Termination Obstacles

Saturation may not reach fixpoint. The canonical pathology is the combination of associativity

$$\text{add}(\text{add}(x, y), z) \approx \text{add}(x, \text{add}(y, z))$$

and commutativity

$$\text{add}(x, y) \approx \text{add}(y, x).$$

Together they generate every parenthesisation of every permutation, which is infinite. Practical implementations therefore:

- Cap iteration count and total e-node count.
- **Schedule** rules: cheap rules apply every iteration, costly ones (notably AC laws) apply less often, or only after the cheap rules saturate.
- Use AC-aware matching: treat `add` as an associative-commutative operator, store its arguments as a multiset, and reason modulo AC rather than enumerating permutations.

4.4 Soundness vs. Completeness

Soundness. Every equality the e-graph claims is genuinely $\approx_R$ -derivable. Two e-nodes occupy a common e-class only because of (a) initial hash consing (syntactic identity), (b) explicit unions from rewrites $l \rightarrow r$ where $l \approx_R r$ holds by definition, or (c) congruence closure (semantic identity in any congruence). All three preserve $\approx_R$.

Completeness. Under unbounded saturation in a system that does terminate, the e-graph captures every $\approx_R$ -equivalence reachable from the initial terms. With a budget, it is an under-approximation: some derivable equalities may be absent. Crucially, under-approximation is safe — everything in one e-class is genuinely equal, even if the e-graph is missing other equalities. We never lose semantic correctness, only optimisation opportunities.

4.5 Worked Example

Saturate with rules $\{\rho_1, \rho_2, \rho_3\}$:

$$\begin{aligned} \rho_1 &: \text{map } (\lambda x. x) \ xs \rightarrow xs, \\ \rho_2 &: \text{reverse } (\text{reverse } xs) \rightarrow xs, \\ \rho_3 &: \text{length } (\text{repeat } x \ n) \rightarrow n, \end{aligned}$$

on the term $t_0 = \text{length } (\text{reverse } (\text{reverse } (\text{repeat } 0 \ 5)))$.

Initial e-graph:

C0:	0	
C1:	5	
C2:	<code>repeat (C0, C1)</code>	
C3:	<code>reverse (C2)</code>	
C4:	<code>reverse (C3)</code>	
C5:	<code>length (C4)</code>	<code><- root</code>

Iteration 1. Search for matches: ρ_2 matches at C_4 (the outer reverse of an inner reverse), with $\theta = \{xs \mapsto C_2\}$. Apply: union C_4 with C_2 . (Rules ρ_1 does not match: no map node yet. ρ_3 does not match: C_5 's argument is C_4 , not in the same e-class as C_2 until the union just performed.)

After rebuild, the e-class structure becomes:

```
C0: 0
C1: 5
{C2, C4}: repeat(C0, C1), reverse(C3)
C3: reverse({C2, C4})
C5: length({C2, C4})
```

Iteration 2. Now $C_5 = \text{length}(\{C_2, C_4\})$, and the class $\{C_2, C_4\}$ contains $\text{repeat}(C_0, C_1)$. Therefore ρ_3 matches at C_5 with $\theta = \{x \mapsto C_0, n \mapsto C_1\}$. Apply: the RHS is $C_1 = 5$. Union C_5 with C_1 .

Iteration 3. No new matches; saturation reached.

Final state: the root e-class contains both the original $\text{length}(\text{reverse}(\text{reverse}(\text{repeat}(0, 5))))$ expression and the literal 5. Extraction (next section) will pick the literal.

5 Extraction

5.1 Problem Statement

Given a saturated e-graph E and a designated **root e-class** r , choose one e-node from each reachable e-class such that the resulting term has minimum cost. Equivalently, find a selection $\text{sel} : \mathcal{C} \rightarrow \text{nodes}(E)$ with $\text{sel}(c) \in c$ for every c , that minimises the cost of the term obtained by following sel recursively from r .

5.2 Tree Cost

Assume an additive cost function $w : F \rightarrow \mathbb{R}_{\geq 0}$ on operators. The cost of a term is

$$\text{cost}(f(t_1, \dots, t_k)) = w(f) + \sum_i \text{cost}(t_i).$$

Lifted to e-graphs:

$$\text{cost}(\text{e-node } (f, [c_1, \dots, c_k])) = w(f) + \sum_i \text{cost}(c_i), \quad \text{cost}(c) = \min_{n \in c} \text{cost}(n).$$

5.3 Fixpoint Algorithm

```
for c in classes: cost[c] <- infinity
repeat until no change:
  for c in classes:
    for n = (f, [c1, ..., ck]) in c:
      candidate <- w(f) + sum(cost[ci] for ci in children)
      if candidate < cost[c]:
        cost[c] <- candidate
    best[c] <- n
```

Termination. Cost values decrease monotonically and are bounded below by zero (or by $\min_f w(f)$ for non-leaf classes), so only finitely many iterations occur.

Correctness. The final values satisfy the recurrence $\text{cost}(c) = \min_n (w(f) + \sum_i \text{cost}(c_i))$. Reading off the chosen e-node $\text{best}(c)$ at the root and recursing yields a term of cost $\text{cost}(r)$, which by construction is the minimum tree cost.

Cycles. E-classes may be cyclic. For example, the rule $\rho : x \rightarrow \text{add}(x, 0)$ adds $\text{add}(x, 0)$ to the e-class of x , where the first child is the same e-class. Initialising $\text{cost} = \infty$ and relaxing handles this gracefully: the cyclic e-node gives $\text{cost } \infty + 1 = \infty$ in the first iteration, but any non-cyclic e-node in the same class gives a finite cost, breaking the cycle. The fixpoint converges on a non-cyclic selection.

5.4 DAG Cost is NP-hard

Tree cost double-counts shared sub-terms. A more realistic cost model — count each distinct sub-term once — is **DAG cost**, where the cost is the sum of $w(f)$ over a *set* of e-nodes (one per chosen e-class) rather than a sum over the tree. Optimising DAG cost is NP-hard via reduction from set cover: each e-node “covers” the e-classes of its chosen children, and minimising the total weight of chosen e-nodes such that the root is reachable is exactly weighted set cover. Practical implementations (notably **egg**) provide an integer-linear-programming extractor; recent algorithmic work has produced faster approximations and exact methods for special cases.

For our purposes (size-minimisation in a DSL where sub-term sharing is uncommon), tree-cost extraction with the fixpoint algorithm is sufficient.

5.5 Worked Example

Consider the e-graph after iteration 2 of §4.5, with the root e-class containing both the literal 5 and the original length expression. Take $w(\text{leaf}) = 1$ and $w(\text{op}) = 1$ (matching the size metric of §2.1 of the enumeration document).

Iteration	C_0	$C_1 \cup C_5$	$C_2 \cup C_4$	C_3
0	∞	∞	∞	∞
1	1	1 (literal 5)	∞	∞
2	1	1	$1 + 1 + 1 = 3$	∞
3	1	1	3	$1 + 3 = 4$
4	1	1	3	4 (no change)

The root e-class has cost 1, achieved by selecting the literal 5. The original program of size 5 has been quotiented to a literal of size 1.

6 Application to the MLMP DSL

6.1 The DSL Recap

The DSL (cf. §1 of the enumeration document) is a simply-typed lambda calculus with conditionals over a fixed set of primitives. The full primitive set, taken from `src/lang/grammar.py`, is:

- *Arithmetic and predicates:* `add`, `subtract`, `multiply`, `divide`, `modulo`, `less_than`, `greater_than`, `equals`, `boolean_and`, `boolean_or`, `boolean_not`, `is_even`, `is_odd`.

- *List constructors and indexing:* singleton, repeat, range, cons, append, concat, first, second, third, last, nth.
- *List manipulation:* insert, replace, swap, cut_idx, cut_val, cut_vals, drop, droplast, take, takelast, slice, cut_slice, splice, zip, is_in.
- *Aggregation and structural:* length, max, min, product, sum, reverse, flatten, unique.
- *Higher-order:* map, mapi, filter, filteri, fold, foldi, count, find, sort, group.

All numeric values are computed modulo 100 (the project convention). Rewrites involving constant folding must respect this.

6.2 A Candidate Rewrite System R_{MLMP}

We propose the following equational laws, organised by category. Each line carries a side condition column where applicable; the absence of one means the rule applies unconditionally to type-correct matches.

LHS	RHS	Side condition
<i>Identity laws</i>		
<code>map (λx. x) xs</code>	<code>xs</code>	—
<code>filter (λx. ⊤) xs</code>	<code>xs</code>	—
<code>concat xs []</code>	<code>xs</code>	—
<code>concat [] xs</code>	<code>xs</code>	—
<code>add x 0</code>	<code>x</code>	second arg literal 0
<code>multiply x 1</code>	<code>x</code>	second arg literal 1
<i>Annihilators</i>		
<code>multiply x 0</code>	<code>0</code>	second arg literal 0
<code>filter (λx. ⊥) xs</code>	<code>[]</code>	—
<code>map f []</code>	<code>[]</code>	—
<i>Involutions</i>		
<code>reverse (reverse xs)</code>	<code>xs</code>	—
<code>boolean_not (boolean_not b)</code>	<code>b</code>	—
<i>Constant folding</i>		
<code>length (repeat x n)</code>	<code>n</code>	—
<code>product (repeat c n)</code>	<code>cⁿ mod 100</code>	both <i>c, n</i> literal
<code>sum (repeat c n)</code>	<code>(c · n) mod 100</code>	both <i>c, n</i> literal
<code>reverse (repeat x n)</code>	<code>repeat x n</code>	—
<code>first (repeat x n)</code>	<code>x</code>	$n \geq 1$
<i>Conditional collapse</i>		
<code>if ⊤ then a else b</code>	<code>a</code>	—
<code>if ⊥ then a else b</code>	<code>b</code>	—
<code>if c then a else a</code>	<code>a</code>	—
<i>Fusion</i>		
<code>map f (map g xs)</code>	<code>map (λx. f (g x)) xs</code>	—
<code>filter p (filter q xs)</code>	<code>filter (λx. p x ∧ q x) xs</code>	—
<code>reverse (map f xs)</code>	<code>map f (reverse xs)</code>	—
<i>Length distribution</i>		
<code>length (concat xs ys)</code>	<code>add (length xs) (length ys)</code>	—
<code>length (map f xs)</code>	<code>length xs</code>	—
<code>length (reverse xs)</code>	<code>length xs</code>	—

This is a starting set, not a final one. The rule set should be tuned empirically against the corpus by measuring how many programs are simplified and by how much. Any rewrite that ever produces a fingerprint mismatch with its LHS on the test suite is unsound and must be rejected.

6.3 Cost Function

Use the size metric of §2.1 of the enumeration document: $w(\text{leaf}) = 1$, $w(f) = 1$ for every internal operator (including λ and if). The total tree cost of a term is precisely its AST size. Consequently, the fixpoint extractor of §5.3 returns the smallest representation of the root e-class under tree cost, with no further tuning required.

6.4 Type-correctness of Rewrites

A rewrite $l \approx r$ is type-correct iff l and r have the same principal type under the same typing context Γ . The DSL’s polymorphic primitives (cf. §1.2 of the enumeration document) are stored in R_{MLMP} as polymorphic schemas; at e-match time, the type substitution is recovered from the matched e-class and the rewrite is instantiated accordingly. Concretely, **reverse** (**reverse** xs) $\approx xs$ holds for any $xs : \text{list}[\sigma]$, and is matched against e-classes of any list type without duplication of the rule.

6.5 Integer Holes

The pipeline operates on *sketches* containing integer holes $\square_{\mathbb{Z}}$ (cf. §2.4 of the enumeration document). Rewrites that pattern-match on integer literals must include a side condition that excludes holes:

Rewrite	Hole-aware side condition
add $x \ 0 \rightarrow x$	second argument is literal 0, not a hole
multiply $x \ 0 \rightarrow 0$	likewise
multiply $x \ 1 \rightarrow x$	likewise
product (repeat $c \ n$) $\rightarrow c^n \bmod 100$	both c, n are literals, neither a hole

A rewrite that ignores this side condition would incorrectly fire on **add** $x \ \square_{\mathbb{Z}}$, rewriting it to x . The eventual instantiation of $\square_{\mathbb{Z}}$ is unknown, and could be any value in C_{seed} — typically not zero. The rewrite is therefore unsound on sketches.

The cleanest treatment is to perform equality saturation *after* hole instantiation, on concrete programs only. This preserves the simplicity of the rewrite system at the cost of running saturation per concrete program. An alternative is to extend R_{MLMP} with hole-aware rules (e.g. “**add** $x \ \square_{\mathbb{Z}}$ is irreducible”), which is more efficient at scale but harder to verify.

6.6 Worked End-to-end Example

Consider the synthetic but plausible RL output:

$$t_0 = \text{length} (\text{reverse} (\text{concat} (\text{map} (\lambda x. \text{add } x \ 0) \ xs) (\text{repeat } 8 \ 10))).$$

Its size is 13: one each for **length**, **reverse**, **concat**, **map**, λ , **add**, x , 0, xs , **repeat**, 8, 10, plus the lambda body’s outer node count.

Initial e-graph (abbreviated):

C0: var x	C1: 0	C2: 8	C3: 10	C4: var xs
C5: add (C0, C1)				
C6: lambda (x , C5)				
C7: map (C6, C4)				
C8: repeat (C2, C3)				
C9: concat (C7, C8)				
C10: reverse (C9)				
C11: length (C10)				
				<- root

Iteration 1. The rule $\rho_{\text{add0}} : \text{add } y \ 0 \rightarrow y$ matches C_5 with $\theta = \{y \mapsto C_0\}$. Union C_5 with C_0 .

Iteration 2. After rebuild, the lambda body’s e-class is $\{C_0, C_5\}$, which contains the bound variable x . Therefore $\rho_{\text{map-id}} : \text{map } (\lambda x. x) \ xs \rightarrow xs$ matches C_7 with $\theta = \{xs \mapsto C_4\}$. Union C_7 with C_4 .

Iteration 3. $C_9 = \text{concat}(\{C_4, C_7\}, C_8)$. The length-of-concat law $\rho_{\text{len-concat}}$ applies at C_{11} (after C_{10} , but the length-of-reverse law $\rho_{\text{len-rev}}$ first reduces $\text{length}(\text{reverse } xs) \approx \text{length } xs$, which adds $\text{length}(C_9)$ to the root e-class). So union C_{11} with the new $\text{length}(C_9)$ class. Then $\rho_{\text{len-concat}}$ fires: $\text{length}(\text{concat } xs \ ys) \rightarrow \text{add}(\text{length } xs, \text{length } ys)$, adding $\text{add}(\text{length}\{C_4, C_7\}, \text{length}(C_8))$ to the root e-class.

Iteration 4. The rule $\rho_{\text{len-rep}} : \text{length}(\text{repeat } x \ n) \rightarrow n$ matches at $\text{length}(C_8)$, unioning that e-class with $C_3 = 10$.

Iteration 5. Saturated.

Extraction. Selected entries from the cost table:

e-class	best representative	cost
$\{C_0, C_5\}$	$\text{var } x$	1
$\{C_4, C_7\}$	$\text{var } xs$	1
$\text{length}(\{C_4, C_7\})$	$\text{length}(xs)$	2
$\text{length}(C_8)$ (joined with C_3)	literal 10	1
root e-class	$\text{add}(\text{length}(xs), 10)$	4

Extracted output:

$$\mathcal{Q}(t_0) = \text{add } (\text{length } xs) \ 10,$$

of size 4.

Verification. On every input xs , the original program computes $\text{length}(\text{reverse}(\text{concat}(xs', \text{repeat}(8, 10))))$ where $xs' = \text{map}(\lambda x. x + 0, xs) = xs$. Therefore the value is $|xs| + 10$, which is exactly what the simplified program computes. By computing fingerprints on the 10-input test suite of the enumeration document, $\Phi(t_0) = \Phi(\mathcal{Q}(t_0))$ holds entry-wise, confirming behaviour preservation. The simplification ratio is $13/4 \approx 3.3\times$.

7 Where This Fits in the Pipeline

7.1 Two Notions of Equivalence

The pipeline currently uses one notion of equivalence; this document introduces a second.

Observational equivalence ($\approx_T$). $p \approx_T q$ iff $\Phi(p) = \Phi(q)$ on the test suite T . This is what §2.2 of the enumeration document calls observational equivalence. It is sound for diversity claims about the corpus (with rare false-positive collisions on the test suite), and crucially it operates only on *root behaviour* — a program’s internal structure is invisible to it.

Equational equivalence ($\approx_E$). $p \approx_E q$ iff p and q are equal under the equational theory of R_{MLMP} . This is the subject of this document. It is sound under E (every equality is provably valid), incomplete in general (semantic equalities not derivable from E are missed), and operates on internal structure — it can detect that a sub-tree is constant or that two structurally distinct sub-trees compute the same value when E knows the relevant law.

7.2 Hybrid Usage

The natural integration is to apply $\mathcal{Q}$ as a normaliser *before* fingerprinting:

1. After enumeration or RL sampling, replace every program p with its extraction $\mathcal{Q}(p)$.
2. Compute $\Phi(\mathcal{Q}(p))$; deduplicate against the running `corpus_fingerprints` table from §3.5 of the enumeration document.
3. The corpus shrinks because programs that differ only in compressible sub-trees (the `product (repeat 10 8)` example) collapse to the same canonical form.

This composition has three desirable properties. First, every retained program is in canonical form, so the corpus is more compact and downstream training trajectories are shorter. Second, the diversity guarantee is preserved: if $\Phi(\mathcal{Q}(p)) \neq \Phi(\mathcal{Q}(q))$ on the test suite, then certainly $p \not\approx_T q$. Third, since $\mathcal{Q}$ is a function, two programs that simplify to the same canonical form are guaranteed to be deduplicated — a stronger property than observational equivalence, which only catches programs that happen to agree on T .

7.3 Sanity-checking Rewrites

Every rule $l \rightarrow r$ in R_{MLMP} should be validated against the test suite before deployment: instantiate l on a randomised set of probe values, evaluate both sides, and reject the rule if they ever disagree. This catches bugs in hand-written rules (e.g. wrong argument order, missing side conditions, off-by-one) without requiring a formal proof of soundness for each rule. The cost is a one-off validation pass; the benefit is confidence that $\mathcal{Q}$ never silently corrupts behaviour.

7.4 Cost Considerations

Equality saturation is more expensive than the existing fingerprint-and-hash deduplication. E-matching is the bottleneck (potentially exponential), and the e-graph itself can grow large during saturation. Mitigations:

- Bound the saturation by node count and iteration count.
- Run only on programs above a size threshold; small programs are unlikely to contain compressible sub-trees.
- Parallelise across programs (each saturation is independent).
- Cache canonical forms by AST hash; identical programs need only be saturated once.

Empirical measurement on a sample of the corpus is the right way to choose budgets; we leave specific numbers to a future implementation.

8 Adjacent Techniques

Knuth–Bendix completion (Knuth and Bendix, 1970) is the procedure described in §1.5. When it succeeds, it gives a confluent and terminating rewrite system, and rewriting to normal form is a complete decision procedure for $\approx_E$. It complements equality saturation: where

completion succeeds, no e-graph is needed; where it fails (notably under AC laws), e-graphs are the pragmatic alternative.

Superoptimisation (Massalin, 1987, Schkufza et al., 2013) searches for a shorter program equivalent to a given one, using SMT solvers or stochastic search rather than directed rewriting. STOKE in particular uses MCMC sampling over candidate programs, accepting moves that decrease cost without changing observable behaviour. This is a generalisation of equality saturation in the sense that it allows arbitrary equivalences (not only those derivable from a fixed rule set), at the cost of much more expensive verification per step.

Partial evaluation (Jones et al., 1993) specialises a program to known inputs by aggressively unfolding and constant-folding. The classical Futamura projections relate partial evaluation to compilation. Pure constant folding (e.g. `product (repeat 10 8) → value`) is a special case; equality saturation subsumes it when the relevant evaluation rules are added to R .

Abstract interpretation (Cousot and Cousot, 1977) computes sound over-approximations of program behaviour by interpreting the program in an abstract domain (e.g. intervals, polyhedra, sign domains). It does not rewrite the program but *licenses* rewrites: knowing that a sub-tree always evaluates to a constant or to a value in a known range justifies replacing it with that constant or with a simpler form. In a fuller pipeline, abstract interpretation could discover side conditions (“ $n \geq 1$ here”) that enable rewrites otherwise blocked by uncertainty.

Translation validation verifies after the fact that two programs are observationally equivalent, without committing to a specific rewriting strategy. It is most useful when the optimiser is a black box: given input and output, prove they agree. We do not need this in the present setting — our optimiser is precisely $\mathcal{Q}$, whose soundness follows from the soundness of R_{MLMP} — but it is a useful safety net for compositions of multiple optimisation passes.

9 Summary

The four pieces compose as follows. An **equational theory** E specifies which programs are considered equal — it is the user-facing definition of program equivalence beyond mere syntactic identity. An **e-graph** is a data structure that compactly represents a set of equivalence classes of terms, maintained under hash consing and congruence closure. **Equality saturation** is the algorithm that grows the e-graph by repeatedly applying every rule in the oriented presentation of E , to fixpoint or to a budget. **Extraction** is the optimisation pass that picks one representative per e-class minimising a chosen cost, returning a single canonical term per equivalence class.

Decoupling *what is equal* (the rules) from *what is good* (the cost) is the central design win. We can specify equational laws independently of the cost we want to minimise — size, latency, numerical accuracy, register pressure — and tune each independently. We never have to commit to a directional reading of any equation, sidestepping the phase-ordering pitfalls of naive term rewriting.

Precisely: under tree cost, the extractor returns the e-node in the root e-class with minimum cost *among representations realised in the e-graph*. With unbounded saturation in a confluent and terminating R , this is globally minimum; with a bounded saturation, it is the best in the explored subset — still sound, possibly suboptimal.

Returning to the open problem from §A.3 of the enumeration document: equality saturation, applied with a rewrite system like R_{MLMP} from §6.2, addresses precisely the redundancy described there. The `product (repeat 10 8)` example reduces to a literal in one iteration. More elaborate cases — map fusion, identity collapse, length distribution — compose via the e-graph without phase-ordering hazards.

A A Second Worked Example

Take a longer program with several layered redundancies:

$$t'_0 = \text{sum}(\text{map}(\lambda x. \text{multiply } x \ 1) (\text{filter}(\lambda y. \top) (\text{reverse}(\text{reverse}(\text{map}(\lambda z. z) \ xs))))).$$

Size: 17. Apply rewrites:

- $\rho_{\text{mul1}} : \text{multiply } x \ 1 \rightarrow x$ at the inner lambda.
- $\rho_{\text{filter-true}} : \text{filter}(\lambda y. \top) \ ys \rightarrow ys$ at the filter.
- $\rho_{\text{rev2}} : \text{reverse}(\text{reverse } ys) \rightarrow ys$ at the double reverse.
- $\rho_{\text{map-id}} : \text{map}(\lambda z. z) \ ys \rightarrow ys$ at both the inner and the (post-collapse) outer map.

Iteration trace:

Iter	Effect
1	ρ_{mul1} unions the body of the outer λx with x .
2	After rebuild, the outer map's lambda has body equivalent to its bound variable. $\rho_{\text{map-id}}$ matches the c
2	In parallel, ρ_{rev2} fires on the double reverse, unioning with the inner $\text{map}(\lambda z. z, xs)$.
2	$\rho_{\text{filter-true}}$ fires on the filter, unioning with the (now-collapsed) reverse argument.
3	$\rho_{\text{map-id}}$ fires on the inner $\text{map}(\lambda z. z, xs)$, unioning with xs .
4	After rebuild, the entire filter/reverse/reverse/map chain collapses by congruence into the e-class c
5	Saturation.

Extraction selects $\text{sum}(xs)$ from the root e-class — size 2. Reduction by $17/2 = 8.5\times$.

This illustrates how rewrites cascade: each rule fires opens up further matches in the next iteration, and after saturation the entire chain of nested redundancies has been collapsed by a combination of explicit rewrites and free congruence-driven unions on parent contexts.

References

- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1977. *In sec. 8.*
- Neil D Jones, Carsten K Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993. *In sec. 8.*
- Donald E Knuth and Peter B Bendix. Simple word problems in universal algebras. In John Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, 1970. *In sec. 1.5 and 8.*
- Henry Massalin. Superoptimizer: A look at the smallest program. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 1987. *In sec. 8.*
- Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2013. *In sec. 8.*

Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021. *In sec.*
4.2.